Spring 2026 ML4P, Problem Set #2

Symmetries & Unsupervised Learning

Spring 2026

Submission Policy

- Submit your code via a link to a git repository (e.g. `GitHub`) on Brightspace.
- If you collaborate with anyone and/or use online/textbook resources, you must cite them. You are allowed to use AI chatbots, but you must cite them.
- You are graded on effort, not correctness. To receive full credit, you must make a genuine attempt at each problem and clearly explain your reasoning, even if your final answer is wrong. This means just submitting raw chatbot dialogue will not count as full credit.

Datasets

- Problem 2 uses the *TheWell* dataset which is a collection of high quality physics simulations. See https://polymathic-ai.org/the_well/
- Problem 3 uses the the same data as Problem Set 1. See https://www.sdss4.org/dr17/irspec/.

1 Warmup: $\mathbb{Z}_2$ Symmetry

Consider a function which has the $\mathbb{Z}_2$ symmetry, that is you have an even function $f(x) = f(-x)$.

1. Create a neural network h_θ which is restricted to learning even functions. Describe it mathematically, and prove that your architecture satisfies the $\mathbb{Z}_2$ symmetry.
2. Implement your architecture on a couple suitable toy problems. Does it out perform the baseline (e.g. a comparable architecture w/ the symmetry constraint removed)?

2 Designing Symmetry-Respecting Architectures

In this course you've seen how architectural choices can restrict the hypothesis class to functions that respect certain symmetries—translation equivariance in CNNs, permutation invariance in DeepSets, etc. In this problem, you will design, justify, and test your own symmetry-respecting architecture.

For this problem, we'll be emulating simulations of PDEs common to physics. That is, given a snapshot of the field $\varphi(t_i, x)$ at a time t_i, predict the configuration at a time in the future $\varphi(t_{i+1}, x)$. To prevent everyone from writing their own simulation, we'll use an open-source package called *TheWell* (see documentation at https://polymathic-ai.org/the_well/) which allows you to download field configurations as a PyTorch Dataset object. Some of these datasets are huge ($\sim$ TB) so choose something reasonable for your computer, OR you can also download a subset of a dataset. Here's a brief example of how to download the data, or you can go to their GitHub / HuggingFace.

Part 1: Choose a Symmetry and Task

Clearly state:

(a) What dataset (type of physics simulation) will you use? Describe the simulation's setup (e.g. boundary conditions, dimension of data, number of data points).

(b) What symmetry or property are you enforcing? Why is this natural for your chosen dataset (1-2 sentences)?

Part 2: Construct the Architecture

Design an architecture that respects your chosen symmetry.

(a) **Construction:** Describe your architecture mathematically. Define the forward pass (equivariant linear layer, local pooling, global pooling), specifying how inputs map to outputs at each layer.

Trivial constructions (e.g., making a network invariant by discarding the input) will not receive full credit. Your architecture should be expressive enough to be useful for the task.

(b) **Proof of property:** Prove (or give a good argument) that your architecture satisfies the claimed property. For example:

- If claiming G-invariance, show that $f(\rho_{\text{in}}(g)\, x) = f(x)$ for all $g \in G$.
- If claiming G-equivariance, show that $f(\rho_{\text{in}}(g)\, x) = \rho_{\text{out}}(g)\, f(x)$.

State any assumptions (e.g., restrictions on activation functions, weight constraints).

Bonus: Prove your architecture can approximate all functions within the proposed function class. This might be very non-trivial.

You are welcome to build on published architectures (e.g., implement a known equivariant layer from a paper), but you must write the proof yourself and clearly cite your sources.

Part 3: Experiments

Implement your architecture and compare it against an unconstrained baseline: the same (or comparable) architecture with the symmetry constraint removed. For example, if you build an $SO(2)$-equivariant layer, your baseline could be an ordinary dense layer or CNN with the same parameter budget.

(a) **Implementation:** Provide clean, runnable code which implements your proposed architecture. Make it easy for the grader to reproduce your results; e.g. include a README or notebook that reproduces your results.

(b) **Sanity Check:** Convince the grader your implementation is correct. E.g. do some unit test / functional tests which show the symmetry is working as intended.

(c) **Comparison:** Train both models on the same task and report the relevent metric (as described in TheWell). Models with symmetries are notoriously difficult to train, so what tricks did you implement? Did these tricks subtly break the symmetry? Is your model roboust to multiple seeds?

(d) **Discussion:** Interpret your results. Here are a couple questions I'd ask:

- Does the constrained architecture learn faster or generalize better, especially in the low-data regime?
- If the constrained model does *not* outperform the baseline, why might that be? (This is fine. An architecture with thoughtful explanation of negative results receives full credit.)

3 Unsupervised Learning:

Left for Hogg: We talked about something reusing the spectral data but is a missing data problem. Compare PCA to a more fancy technique?